

Data Visualization & Correlation Analysis

Comprehensive EDA with GSS Dataset

Instructor: Lumumba Wandera Victor

Mediacrest Training College

20th April 2026

Learning Objectives

- Master univariate visualizations for categorical and continuous variables
- Create compelling bivariate plots to explore relationships
- Build multivariate visualizations for complex pattern detection
- Understand and visualize correlation matrices
- Perform comparative analysis across demographic groups
- Interpret visualizations to extract actionable insights
- Apply best practices in statistical graphics

Dataset Overview & Setup

GSS Subset Dataset

Records: 2,538 respondents

Variables: 9 columns

Types: Mixed (categorical + continuous)

```
1 df = pd.read_csv(  
2 'GSSsubset.csv') sns.set_style("whitegrid") plt.rcParams['figure.figsize'] = [10, 6]
```

1. Univariate Analysis: Distribution of Income

```
1 plt.figure(figsize=(10, 6))
2 sns.histplot(df['income'], bins=50, kde=True,
3             color='skyblue', edgecolor='black')
4 plt.title('Distribution of Income', fontsize=14, fontweight='bold')
5 plt.xlabel('Income ($)')
6 plt.ylabel('Frequency')
7 plt.axvline(df['income'].median(), color='red',
8            linestyle='--', label=f'Median: ${df["income"].median():.0f}')
9 plt.legend()
10 plt.show()
11
```

Income Distribution: Box Plot Analysis

```
1 plt.figure(figsize=(8, 6))
2 sns.boxplot(y=df['income'], color='lightcoral')
3 plt.title('Income Distribution - Box Plot', fontsize=14, fontweight='bold')
4 plt.ylabel('Income ($)')
5 plt.axhline(df['income'].mean(), color='blue',
6             linestyle='--', alpha=0.7, label='Mean')
7 plt.axhline(df['income'].median(), color='green',
8             linestyle='--', label='Median')
9 plt.legend()
10 plt.show()
11
12 print(f"Outliers: {len(df[df['income'] > df['income'].quantile(0.75) +
13                        1.5*(df['income'].quantile(0.75)-df['income'].quantile(0.25))])}")
14
```

Age Distribution: Multiple Visualizations

```
1 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
2
3 sns.histplot(df['age'], bins=20, kde=True, ax=axes[0],
4             color='steelblue', edgecolor='black')
5 axes[0].set_title('Age Distribution - Histogram', fontweight='bold')
6 axes[0].axvline(df['age'].mean(), color='red', linestyle='--')
7
8 sns.violinplot(y=df['age'], ax=axes[1], color='lightgreen')
9 axes[1].set_title('Age Distribution - Violin Plot', fontweight='bold')
10 axes[1].axhline(df['age'].mean(), color='red', linestyle='--')
11
12 plt.tight_layout()
13 plt.show()
14
```

Height and Weight: Joint Distribution

```
1 sns.jointplot(data=df, x='height', y='weight',  
2               kind='scatter', alpha=0.6, color='purple',  
3               marginal_ticks=50)  
4 plt.suptitle('Height vs Weight Distribution', y=1.02,  
5               fontsize=14, fontweight='bold')  
6 plt.show()  
7  
8 corr = df['height'].corr(df['weight'])  
9 print(f"Height-Weight Correlation: {corr:.3f}")  
10
```

2. Categorical Variables: Gender Distribution

```
1 plt.figure(figsize=(8, 6))
2 ax = sns.countplot(data=df, x='sex', palette='Set2',
3                     edgecolor='black', linewidth=1.5)
4 plt.title('Gender Distribution', fontsize=14, fontweight='bold')
5 plt.xlabel('Gender')
6 plt.ylabel('Count')
7
8 total = len(df)
9 for p in ax.patches:
10     percentage = f'{100 * p.get_height() / total:.1f}%'
11     x = p.get_x() + p.get_width() / 2
12     y = p.get_height() + 10
13     ax.annotate(percentage, (x, y), ha='center', fontweight='bold')
14 plt.show()
15
```

Education Level Distribution

```
1 plt.figure(figsize=(10, 6))
2 degree_counts = df['degree'].value_counts()
3 ax = sns.barplot(x=degree_counts.values, y=degree_counts.index,
4                 palette='viridis', edgecolor='black')
5 plt.title('Education Level Distribution', fontsize=14, fontweight='bold')
6 plt.xlabel('Count')
7 plt.ylabel('Degree Level')
8
9 for i, v in enumerate(degree_counts.values):
10     ax.text(v + 10, i, str(v), va='center', fontweight='bold')
11 plt.tight_layout()
12 plt.show()
13
```

Marital Status: Pie Chart & Count Plot

```
1 fig, axes = plt.subplots(1, 2, figsize=(14, 6))
2
3 marital_counts = df['marital'].value_counts()
4 colors = plt.cm.Set3(np.linspace(0, 1, len(marital_counts)))
5 axes[0].pie(marital_counts.values, labels=marital_counts.index,
6             autopct='%1.1f%%', colors=colors, startangle=90,
7             explode=[0.05]*len(marital_counts))
8 axes[0].set_title('Marital Status - Pie Chart', fontweight='bold')
9
10 sns.countplot(data=df, x='marital', ax=axes[1],
11               palette='pastel', edgecolor='black')
12 axes[1].set_title('Marital Status - Count Plot', fontweight='bold')
13 axes[1].tick_params(axis='x', rotation=45)
14 plt.tight_layout()
15 plt.show()
16
```

3. Bivariate Analysis: Income by Gender

```
1 plt.figure(figsize=(10, 6))
2 sns.boxplot(data=df, x='sex', y='income', palette='Set1')
3 plt.title('Income Distribution by Gender', fontsize=14, fontweight='bold')
4 plt.xlabel('Gender')
5 plt.ylabel('Income ($)')
6
7 means = df.groupby('sex')['income'].mean()
8 plt.scatter(range(len(means)), means.values, color='red',
9             s=100, zorder=10, marker='*', label='Mean')
10 plt.legend()
11 plt.show()
12
13 print(df.groupby('sex')['income'].describe())
14
```

Income by Education Level

```
1 plt.figure(figsize=(12, 7))
2 sns.violinplot(data=df, x='degree', y='income',
3               palette='coolwarm', inner='box')
4 plt.title('Income Distribution by Education Level',
5           fontsize=14, fontweight='bold')
6 plt.xlabel('Education Level')
7 plt.ylabel('Income ($)')
8 plt.xticks(rotation=45, ha='right')
9 plt.tight_layout()
10 plt.show()
11
12 print(df.groupby('degree')['income'].agg(['mean', 'median', 'std']))
13
```

Age vs Income: Scatter Plot with Trend

```
1 plt.figure(figsize=(10, 6))
2 sns.regplot(data=df, x='age', y='income', scatter_kws={'alpha':0.3},
3               line_kws={'color':'red', 'linewidth':2})
4 plt.title('Age vs Income Relationship', fontsize=14, fontweight='bold')
5 plt.xlabel('Age (years)')
6 plt.ylabel('Income ($)')
7
8 corr = df['age'].corr(df['income'])
9 plt.text(0.05, 0.95, f'Correlation: {corr:.3f}',
10          transform=plt.gca().transAxes, fontsize=12,
11          bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
12 plt.show()
13
```

Hours Worked vs Income

```
1 plt.figure(figsize=(10, 6))
2 plt.hexbin(df['hrswrk'], df['income'], gridsize=30,
3           cmap='Blues', mincnt=1)
4 plt.colorbar(label='Count')
5 plt.title('Hours Worked vs Income (Hexbin)',
6         fontsize=14, fontweight='bold')
7 plt.xlabel('Hours per Week')
8 plt.ylabel('Income ($)')
9
10 sns.regplot(data=df, x='hrswrk', y='income', scatter=False,
11            color='red', linewidth=2)
12 plt.show()
13
```

4. Multivariate Analysis: Pair Plot

```
1 cols_to_plot = ['age', 'height', 'weight', 'hrswrk', 'income']
2 sns.pairplot(df[cols_to_plot].dropna(), diag_kind='kde',
3             plot_kws={'alpha':0.5}, corner=True)
4 plt.suptitle('Pair Plot - Continuous Variables', y=1.02,
5             fontsize=14, fontweight='bold')
6 plt.show()
7
```

Pair Plot with Categorical Hue

```
1 cols_subset = ['age', 'income', 'hrswrk', 'weight']
2 sns.pairplot(df, vars=cols_subset, hue='sex',
3               plot_kws={'alpha':0.6}, diag_kind='hist',
4               palette='Set1')
5 plt.suptitle('Pair Plot by Gender', y=1.02,
6             fontsize=14, fontweight='bold')
7 plt.show()
8
```

Correlation Heatmap

```
1 plt.figure(figsize=(10, 8))
2 numeric_cols = ['age', 'height', 'weight', 'hrswrk', 'income']
3 corr_matrix = df[numeric_cols].corr()
4
5 sns.heatmap(corr_matrix, annot=True, cmap='coolwarm',
6             center=0, square=True, linewidths=1,
7             fmt='.3f', cbar_kws={"shrink": .8})
8 plt.title('Correlation Matrix - Continuous Variables',
9           fontsize=14, fontweight='bold')
10 plt.tight_layout()
11 plt.show()
12
```

Income by Gender and Education

```
1 plt.figure(figsize=(12, 7))
2 pivot_data = df.groupby(['degree', 'sex'])['income'].mean().unstack()
3 pivot_data.plot(kind='bar', ax=plt.gca(),
4                 colormap='Set2', edgecolor='black',
5                 width=0.8)
6 plt.title('Average Income by Education and Gender',
7           fontsize=14, fontweight='bold')
8 plt.xlabel('Education Level')
9 plt.ylabel('Average Income ($)')
10 plt.xticks(rotation=45, ha='right')
11 plt.legend(title='Gender')
12 plt.tight_layout()
13 plt.show()
14
```

Stacked Bar Chart: Marital Status by Gender

```
1 plt.figure(figsize=(10, 6))
2 marital_gender = pd.crosstab(df['sex'], df['marital'])
3 marital_gender.plot(kind='bar', stacked=True,
4                     colormap='Pastel1', ax=plt.gca(),
5                     edgecolor='black', linewidth=0.5)
6 plt.title('Marital Status Distribution by Gender',
7           fontsize=14, fontweight='bold')
8 plt.xlabel('Gender')
9 plt.ylabel('Count')
10 plt.xticks(rotation=0)
11 plt.legend(title='Marital Status', bbox_to_anchor=(1.05, 1))
12 plt.tight_layout()
13 plt.show()
14
```

5. Comparative Analysis: Age Distribution by Marital Status

```
1 plt.figure(figsize=(12, 7))
2 sns.boxplot(data=df, x='marital', y='age',
3             palette='Set3', order=df['marital'].value_counts().index)
4 plt.title('Age Distribution by Marital Status',
5           fontsize=14, fontweight='bold')
6 plt.xlabel('Marital Status')
7 plt.ylabel('Age (years)')
8 plt.xticks(rotation=45, ha='right')
9
10 sns.pointplot(data=df, x='marital', y='age',
11              color='red', markers='*', scale=1.5)
12 plt.tight_layout()
13 plt.show()
14
```

Income Distribution Across Marital Status

```
1 plt.figure(figsize=(12, 7))
2 sns.violinplot(data=df, x='marital', y='income',
3               inner='quartile', palette='muted')
4 plt.title('Income Distribution by Marital Status',
5           fontsize=14, fontweight='bold')
6 plt.xlabel('Marital Status')
7 plt.ylabel('Income ($)')
8 plt.xticks(rotation=45, ha='right')
9 plt.tight_layout()
10 plt.show()
11
12 print(df.groupby('marital')['income'].describe())
13
```

Hours Worked by Education and Gender

```
1 g = sns.catplot(data=df, x='degree', y='hrswrk',  
2                 hue='sex', kind='box', height=7, aspect=1.5,  
3                 palette='Set1')  
4 g.set_axis_labels('Education Level', 'Hours per Week')  
5 g.set_titles('Hours Worked by Education and Gender')  
6 plt.xticks(rotation=45, ha='right')  
7 plt.suptitle('Work Hours by Education and Gender',  
8             y=1.02, fontsize=14, fontweight='bold')  
9 plt.tight_layout()  
10 plt.show()  
11
```

6. Advanced Visualization: Income Distribution Faceted by Gender

```
1 g = sns.FacetGrid(df, col='sex', height=5, aspect=1.2)
2 g.map_dataframe(sns.histplot, x='income', bins=40,
3                 kde=True, color='steelblue')
4 g.set_axis_labels('Income ($)', 'Frequency')
5 g.set_titles('Gender: {col_name}')
6 plt.suptitle('Income Distribution by Gender',
7             y=1.03, fontsize=14, fontweight='bold')
8 plt.tight_layout()
9 plt.show()
10
```

Age-Income Relationship by Education Level

```
1 g = sns.lmplot(data=df, x='age', y='income',
2               col='degree', col_wrap=3,
3               scatter_kws={'alpha':0.4, 's':20},
4               line_kws={'color':'red', 'linewidth':2},
5               height=4, aspect=1.2)
6 g.set_axis_labels('Age', 'Income ($)')
7 g.set_titles('Education: {col_name}')
8 plt.suptitle('Age vs Income by Education Level',
9             y=1.02, fontsize=14, fontweight='bold')
10 plt.tight_layout()
11 plt.show()
12
```

7. Distribution Comparisons: KDE Plots

```
1 plt.figure(figsize=(10, 6))
2 for degree in df['degree'].unique():
3     subset = df[df['degree'] == degree]['income']
4     sns.kdeplot(subset, label=degree, linewidth=2)
5
6 plt.title('Income Density by Education Level',
7           fontsize=14, fontweight='bold')
8 plt.xlabel('Income ($)')
9 plt.ylabel('Density')
10 plt.legend(title='Education', bbox_to_anchor=(1.05, 1))
11 plt.tight_layout()
12 plt.show()
13
```

Weight Distribution by Gender - Ridge Plot

```
1 plt.figure(figsize=(10, 8))
2 sns.kdeplot(data=df, x='weight', y='sex',
3             fill=True, common_norm=False,
4             palette='viridis', alpha=0.7)
5 plt.title('Weight Distribution by Gender - Ridge Plot',
6           fontsize=14, fontweight='bold')
7 plt.xlabel('Weight (lbs)')
8 plt.ylabel('Gender')
9 plt.tight_layout()
10 plt.show()
11
```

8. Correlation Analysis: Scatter Matrix with Correlation

```
1 from pandas.plotting import scatter_matrix
2
3 cols = ['age', 'height', 'weight', 'hrswrk', 'income']
4 axes = scatter_matrix(df[cols].dropna(), alpha=0.4,
5                       figsize=(12, 12), diagonal='kde')
6
7 for i, j in zip(*plt.gca().get_axis_gridline_positions()):
8     if i != j:
9         corr = df[cols[i]].corr(df[cols[j]])
10        axes[i, j].annotate(f'r={corr:.2f}',
11                           xy=(0.5, 0.9),
12                           xycoords='axes fraction',
13                           ha='center', va='center',
14                           bbox=dict(boxstyle='round',
15                                   facecolor='white', alpha=0.8))
16
17 plt.suptitle('Scatter Matrix with Correlations',
18             y=0.95, fontsize=14, fontweight='bold')
19 plt.tight_layout()
20 plt.show()
21
```

Partial Correlation: Controlling for Age

```
1 from scipy import stats
2
3 slope, intercept, r_value, p_value, std_err = \
4     stats.linregress(df['age'], df['income'])
5 income_residuals = df['income'] - (intercept + slope * df['age'])
6
7 slope2, intercept2, r_value2, p_value2, std_err2 = \
8     stats.linregress(df['age'], df['hrswrk'])
9 hrswrk_residuals = df['hrswrk'] - (intercept2 + slope2 * df['age'])
10
11 plt.figure(figsize=(10, 6))
12 plt.scatter(hrswrk_residuals, income_residuals, alpha=0.5)
13 plt.axhline(0, color='red', linestyle='--', alpha=0.5)
14 plt.axvline(0, color='red', linestyle='--', alpha=0.5)
15 plt.title('Income vs Hours Worked (Controlling for Age)',
16         fontsize=14, fontweight='bold')
17 plt.xlabel('Hours Worked Residuals')
18 plt.ylabel('Income Residuals')
19 plt.show()
20
```

9. Time-Based Analysis: Age Cohorts

```
1 df['age_group'] = pd.cut(df['age'], bins=[20, 30, 40, 50, 60, 70, 80],
2                           labels=['20-29', '30-39', '40-49',
3                                   '50-59', '60-69', '70-79'])
4
5 plt.figure(figsize=(10, 6))
6 sns.boxplot(data=df, x='age_group', y='income',
7             palette='coolwarm', order=['20-29', '30-39', '40-49',
8                                         '50-59', '60-69', '70-79'])
9 plt.title('Income Distribution by Age Cohort',
10          fontsize=14, fontweight='bold')
11 plt.xlabel('Age Group')
12 plt.ylabel('Income ($)')
13 plt.xticks(rotation=45)
14 plt.tight_layout()
15 plt.show()
16
```

BMI Calculation and Visualization

```
1 df['BMI'] = (df['weight'] * 0.453592) / ((df['height'] * 0.0254) ** 2)
2
3 plt.figure(figsize=(12, 5))
4 plt.subplot(1, 2, 1)
5 sns.histplot(df['BMI'], bins=30, kde=True, color='teal')
6 plt.title('BMI Distribution', fontweight='bold')
7 plt.axvline(df['BMI'].mean(), color='red', linestyle='--',
8             label=f'Mean: {df["BMI"].mean():.1f}')
9 plt.legend()
10
11 plt.subplot(1, 2, 2)
12 sns.boxplot(x='sex', y='BMI', data=df, palette='Set2')
13 plt.title('BMI by Gender', fontweight='bold')
14 plt.tight_layout()
15 plt.show()
16
```

10. Income-BMI Relationship

```
1 plt.figure(figsize=(10, 6))
2 sns.scatterplot(data=df, x='BMI', y='income',
3                 hue='sex', alpha=0.6, s=60)
4 plt.title('Income vs BMI', fontsize=14, fontweight='bold')
5 plt.xlabel('Body Mass Index (BMI)')
6 plt.ylabel('Income ($)')
7
8 plt.axvline(18.5, color='green', linestyle=':', alpha=0.5)
9 plt.axvline(25, color='orange', linestyle=':', alpha=0.5)
10 plt.axvline(30, color='red', linestyle=':', alpha=0.5)
11 plt.text(16, df['income'].max()*0.9, 'Underweight', rotation=90)
12 plt.text(22, df['income'].max()*0.9, 'Normal', rotation=90)
13 plt.text(27, df['income'].max()*0.9, 'Overweight', rotation=90)
14 plt.text(32, df['income'].max()*0.9, 'Obese', rotation=90)
15 plt.legend(bbox_to_anchor=(1.05, 1))
16 plt.tight_layout()
17 plt.show()
18
```

Work Hours Distribution

```
1 plt.figure(figsize=(12, 5))
2 plt.subplot(1, 2, 1)
3 sns.histplot(df['hrswrk'], bins=40, kde=True, color='coral')
4 plt.title('Hours Worked per Week Distribution', fontweight='bold')
5 plt.xlabel('Hours per Week')
6 plt.axvline(df['hrswrk'].mean(), color='blue', linestyle='--',
7             label=f'Mean: {df["hrswrk"].mean():.1f}')
8 plt.axvline(40, color='green', linestyle='--', label='Full-time (40h)')
9 plt.legend()
10
11 plt.subplot(1, 2, 2)
12 sns.boxplot(data=df, x='marital', y='hrswrk', palette='Set3')
13 plt.title('Work Hours by Marital Status', fontweight='bold')
14 plt.xticks(rotation=45, ha='right')
15 plt.tight_layout()
16 plt.show()
17
```

11. Multi-Variable Comparison: Radar Chart

```
1 from math import pi
2 edu_stats = df.groupby('degree')[['age', 'income', 'hrswrk',
3                                   'height', 'weight']].mean()
4 edu_stats_norm = (edu_stats - edu_stats.min()) / (edu_stats.max() -
5             edu_stats.min())
6 categories = list(edu_stats_norm.columns)
7 N = len(categories)
8 angles = [n / float(N) * 2 * pi for n in range(N)]
9 angles += angles[:1]
10
11 fig, ax = plt.subplots(figsize=(8, 8), subplot_kw=dict(projection='polar'))
12 for idx, degree in enumerate(edu_stats_norm.index):
13     values = edu_stats_norm.loc[degree].values.flatten().tolist()
14     values += values[:1]
15     ax.plot(angles, values, linewidth=2, label=degree)
16     ax.fill(angles, values, alpha=0.1)
17
18 ax.set_theta_offset(pi / 2)
19 ax.set_theta_direction(-1)
20 ax.set_thetagrids(np.degrees(angles[:-1]), categories)
21 ax.set_ylim(0, 1)
22 plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))
23 plt.title('Normalized Characteristics by Education',
```

12. Advanced Correlation: Pairwise Correlation with Significance

```
1 from scipy.stats import pearsonr
2 cols = ['age', 'height', 'weight', 'hrswrk', 'income']
3 corr_matrix = pd.DataFrame(index=cols, columns=cols)
4 pval_matrix = pd.DataFrame(index=cols, columns=cols)
5
6 for i in cols:
7     for j in cols:
8         corr, pval = pearsonr(df[i].dropna(), df[j].dropna())
9         corr_matrix.loc[i, j] = f'{corr:.3f}'
10        pval_matrix.loc[i, j] = f'{pval:.3f}'
11
12 print("Correlation Coefficients:")
13 print(corr_matrix)
14 print("\nP-values:")
15 print(pval_matrix)
16
17 plt.figure(figsize=(10, 8))
18 sns.heatmap(df[cols].corr(), annot=True, cmap='RdBu_r',
19             center=0, fmt='.3f', square=True)
20 plt.title('Correlation Matrix with Significance',
21          fontsize=14, fontweight='bold')
22 plt.tight_layout()
```

Income Inequality: Lorenz Curve

```
1 def lorenz_curve(data):
2     sorted_data = np.sort(data)
3     n = len(sorted_data)
4     cumulative_population = np.arange(1, n+1) / n
5     cumulative_income = np.cumsum(sorted_data) / np.sum(sorted_data)
6     return cumulative_population, cumulative_income
7
8 pop, income = lorenz_curve(df['income'].dropna())
9 gini = 1 - 2 * np.trapz(income, pop)
10
11 plt.figure(figsize=(8, 8))
12 plt.plot(pop, pop, 'k--', label='Perfect Equality', linewidth=2)
13 plt.plot(pop, income, 'b-', label=f'Actual (Gini: {gini:.3f})', linewidth=2)
14 plt.fill_between(pop, pop, income, alpha=0.3, color='red')
15 plt.title('Lorenz Curve - Income Inequality', fontsize=14, fontweight='bold')
16 plt.xlabel('Cumulative Population')
17 plt.ylabel('Cumulative Income')
18 plt.legend()
19 plt.grid(True, alpha=0.3)
20 plt.show()
21 print(f"Gini Coefficient: {gini:.3f}")
22
```

13. Distribution Comparison: QQ Plot

```
1 from scipy import stats
2 plt.figure(figsize=(12, 5))
3 plt.subplot(1, 2, 1)
4 stats.probplot(df['income'].dropna(), dist="norm", plot=plt)
5 plt.title('Q-Q Plot: Income', fontweight='bold')
6 plt.xlabel('Theoretical Quantiles')
7 plt.ylabel('Sample Quantiles')
8
9 plt.subplot(1, 2, 2)
10 stats.probplot(df['age'].dropna(), dist="norm", plot=plt)
11 plt.title('Q-Q Plot: Age', fontweight='bold')
12 plt.xlabel('Theoretical Quantiles')
13 plt.ylabel('Sample Quantiles')
14 plt.tight_layout()
15 plt.show()
16
```

Categorical-Numerical Relationship: Point Plot

```
1 plt.figure(figsize=(12, 7))
2 sns.pointplot(data=df, x='degree', y='income',
3               hue='sex', join=True,
4               palette='Set1', markers=['o', 's'],
5               linestyle=['-', '--'])
6 plt.title('Average Income by Education and Gender\n(with 95% CI)',
7           fontsize=14, fontweight='bold')
8 plt.xlabel('Education Level')
9 plt.ylabel('Average Income ($)')
10 plt.xticks(rotation=45, ha='right')
11 plt.legend(title='Gender', bbox_to_anchor=(1.05, 1))
12 plt.tight_layout()
13 plt.show()
14
```

14. Temporal Patterns: Age-Income Lifecycle

```
1 age_income = df.groupby('age')['income'].agg(['mean', 'median',
2         'std']).reset_index()
3 plt.figure(figsize=(12, 6))
4 plt.plot(age_income['age'], age_income['mean'],
5         'b--', linewidth=2, label='Mean Income')
6 plt.fill_between(age_income['age'],
7                 age_income['mean'] - age_income['std'],
8                 age_income['mean'] + age_income['std'],
9                 alpha=0.3, color='blue', label='1 SD')
10 plt.plot(age_income['age'], age_income['median'],
11         'r--', linewidth=2, label='Median Income')
12 plt.title('Income Lifecycle Profile by Age', fontsize=14, fontweight='bold')
13 plt.xlabel('Age (years)')
14 plt.ylabel('Income ($)')
15 plt.legend()
16 plt.grid(True, alpha=0.3)
17 plt.show()
```

Multivariate Distribution: 3D Scatter Plot

```
1 from mpl_toolkits.mplot3d import Axes3D
2 fig = plt.figure(figsize=(10, 8))
3 ax = fig.add_subplot(111, projection='3d')
4 scatter = ax.scatter(df['age'], df['hrswrk'], df['income'],
5                     c=df['BMI'], cmap='viridis',
6                     alpha=0.6, s=30)
7 ax.set_xlabel('Age', labelpad=10)
8 ax.set_ylabel('Hours/Week', labelpad=10)
9 ax.set_zlabel('Income ($)', labelpad=10)
10 plt.title('3D Visualization: Age, Work Hours, Income\n(Color = BMI)',
11          fontsize=14, fontweight='bold', pad=20)
12 cbar = plt.colorbar(scatter, pad=0.1)
13 cbar.set_label('BMI', rotation=270, labelpad=15)
14 plt.tight_layout()
15 plt.show()
16
```

Thank You!

Questions & Discussion

Mediacrest Training College

Instructor: Lumumba Wandera Victor

Date: 20th April 2026

Contact: lumumba.victor@mediacrest.edu

"A picture is worth a thousand numbers"